

系统异常访问监测支持组件

集成指南

标准正式版

适用范围：Spring Boot 2.7.x 与 Spring Boot 3.x 宿主系统

文档状态：正式版

生成日期：2026-07-24

目录

1. 选择依赖与配置	3
最短可用接入	3
2. 实现宿主 SPI 与资源访问记录	4
3. 动作埋点与控制触发	5
3.1 动作编码支持矩阵与控制策略	5
3.2 MVC 注解采集	7
3.3 注册式方法调用埋点	9
3.4 原始草稿与 EventEnricher	10
3.5 规则控制方法绑定	11
4. 内部不可变规则与持久化动态规则	11
5. MyBatis 迁移与告警闭环	12
6. 前端补充信息统一接入	13
6.1 标准请求契约与字段映射	13
6.2 宿主接口适配流程	14
6.3 响应语义、重试与验收	16
7. 初始化插件	16
8. 最小验收、升级与发布	17

本指南面向宿主系统开发者。推荐顺序是：选择 Starter、执行数据库迁移、实现宿主 SPI、以 **OBSERVE** 运行、验收后再启用 **ENFORCE**。完整运行边界与值守项见架构与运维说明。

1. 选择依赖与配置

宿主版本	Starter	Servlet 命名空间
Spring Boot 2.7.x	sys-abnormal-access-monitoring-spring2-starter	javax.servlet
Spring Boot 3.x	sys-abnormal-access-monitoring-spring3-starter	jakarta.servlet

不要同时引入两个 Starter。两者均使用 `abnormal.access.monitor` 前缀：

```
abnormal:
  access:
    monitor:
      system-id: order-service      # 稳定且可追溯的系统标识
      mode: OBSERVE                 # OBSERVE 或 ENFORCE
      frontend:
        enabled: true
      instrumentation:
        enabled: true              # MVC 注解动作采集
      mdc:
        enabled: true              # 将事件 traceId 同步到日志 MDC
        trace-id-key: traceId      # 与宿主日志格式使用同一 MDC 键
      trusted-proxies: [10.0.0.0/8]
```

OBSERVE 仍会记录事件、执行规则、生成告警，但不发起自动控制。Starter 会为每个非 **RECORD** 的 `ControlActionType` 注册一个安全默认回退触发器；它仅在 **ENFORCE** 下命中对应规则动作时写入 **SKIPPED** 控制记录，不会限流、拒绝、锁定、撤销会话或自动审批。默认回退不算可执行的宿主能力，因此 **ENFORCE** 仍只有在至少注册一个真正可执行的宿主 `ControlHandler` 或 `@ControlTrigger` 绑定时才能启动。`trusted-proxies` 仅包含实际受信任的反向代理；来自不受信任直连请求的转发头不能用于确定客户端 IP。

`mdc.enabled` 默认是 `true`。Servlet 请求适配器会把事件的 `traceId` 写入 `mdc.trace-id-key`（默认 `traceId`），供采用 SLF4J MDC 的日志格式关联检索；请求结束时恢复线程原有 MDC。追踪标识优先采用入站追踪头，其次复用当前 MDC，最后由组件生成。消息消费与定时任务不自动绑定 MDC，宿主应自行传播 MDC，并把同一标识传入 `MonitoringRequestContext.traceId(...)`。

最短可用接入

只做 MVC 审计时，客户只需配置 `system-id`，在控制器方法上写一个稳定动作编码；`eventType` 不填即为普通查询：

```
@MonitorAction("order:read")
@GetMapping("/orders/{id}")
public Order find(@PathVariable String id) {
    return orderService.find(id);
}
```

需要自动执行规则控制时，再在任意 Spring Bean 的公开方法上添加 `@ControlTrigger`。无需实现完整 `ControlHandler`、无需手工注册：

```
@Component
class HostControls {
    @ControlTrigger(ControlActionType.RATE_LIMIT)
    public void rateLimit(ControlCommand command) {
        limiter.apply(command.getSubject(), command.getExpiresAt());
    }
}
```

同一 `ControlActionType` 的宿主 `@ControlTrigger` 或 `ControlHandler` 会优先于 Starter 的默认回退触发器，因而可直接替换它。`OBSERVE` 适合先上线观察；只有确认上述方法能安全执行、且幂等键已被宿主处理后才改为 `ENFORCE`。`@MonitorAction` 只记录请求完成后的结果，不替代业务授权或前置阻断。

2. 实现宿主 SPI 与资源访问记录

初始化插件共生成 5 个文件，其中 `host-spi/HostMonitoringSpi.java`、`host-spi/HostMonitoringActions.java` 与 `host-spi/HostControlHandler.java` 需要移入宿主 `src/main/java` 并注册为 Bean。`HostMonitoringSpi` 只包含身份、资源授权与可信代理 SPI；它不包含不会被自动调用的 `EventEnricher`。这些 SPI 不依赖 Spring Security、Sa-Token 或具体权限框架。

SPI	宿主责任
<code>IdentityContextProvider</code>	从服务端已认证上下文取稳定用户 ID、账号类型、角色和会话哈希
<code>ResourceScopeAuthorizer</code>	调用宿主数据权限服务，返回资源级允许或拒绝结论
<code>TrustedProxyResolver</code>	仅在直连地址受信任时解析转发链
<code>EventEnricher</code>	可选补充已批准、非敏感的业务摘要字段；Starter 不会自动发现或调用，宿主须在自己的事件监听/适配位置显式调用
<code>ControlHandler</code>	实现验证码、限流、锁定、撤销会话、二次认证、拒绝或审批流，并按幂等键去重；同动作会覆盖 Starter 默认回退触发器

资源访问使用 `ResourceAccessGuard.authorize(identity, resourceRequest)`。它把宿主授权结果记为标准事件，但监测写入失败不会改变原结论；授权器未配置、返回空结论或抛出异常时一律按拒绝处理。不要根据浏览器参数自行放行资源。

`ControlHandler` 必须按 `ControlCommand.getIdempotencyKey()` 去重，并且只声明实际可执行的动作。例如 `AUTH-02` 的限流目标是 `ip:<sourceIp>`，有效期为 30 分钟；`AUTH-01` 会请求验证码和限流。控制失败不得绕过既有认证与数据权限。控制方法和默认回退触发器都会收到同一份不可变 `ControlCommand`：

参数	说明与宿主使用方式
<code>idempotencyKey</code>	本次控制的稳定去重键；宿主必须以它避免重复写限流、锁定、会话失效或审批任务。
<code>alertId</code>	触发控制的告警标识；用于把宿主处置结果关联回告警闭环。
<code>subject</code>	规则从可信服务端事件得出的受控对象，例如 <code>user:<id></code> 或 <code>ip:<sourceIp></code> ；不得用浏览器传参替换。
<code>action</code>	当前 <code>ControlActionType</code> ；宿主只应执行自己声明支持的动作。

参数	说明与宿主使用方式
<code>expiresAt</code>	规则给出的控制失效时间；限流、挑战、锁定或会话策略应据此设置上限，审批流应据此处理过期。

3. 动作埋点与控制触发

组件以“静态定义统一、动态事实按次补充”为边界提供三条入口，最终都会交给同一个 `SecurityMonitor.record(SecurityEventDraft)`：

入口	适用场景	可携带事实
<code>@MonitorAction</code>	Servlet MVC 控制器的固定业务动作	静态动作定义，以及受限参数路径和调用前后 <code>Enricher</code> 补充的动态事实；最终 HTTP 结果由拦截器补充
<code>MonitoringActionRegistry + ActionEventRecorder</code>	Service、消息消费、批任务，或需要动态资源和业务指标的动作	注册的静态定义，以及每次调用补充的资源 ID、数据量、时延、结果和原因码
原始 <code>SecurityEventDraft</code>	非标准适配器或确实不能使用记录器的集成	调用方必须完整提供所有必填可信字段；优先级最低

3.1 动作编码支持矩阵与控制策略

“宿主必接”表示只要宿主具有该业务能力，就必须从服务端权威位置产生事件；它不是 Starter 的启动前置条件。普通读取以外的资源域拒绝、敏感读取、导出、权限变更和安全运维变更不能仅依赖浏览器遥测。自动注解支持仅限同步 Servlet MVC 的已标注控制器；“需补充”表示动作会自动记录，但只有补齐该列事实才能满足列出的内置规则。

建议 code		内置规则	宿主接入级别	自动注解支持	必需额外事实及采集点	自动控制	审核要求
<code>auth:login-success</code>	LOGIN_SUCCESS	AUTH-02（同 IP 成功/失败比）	认证能力存在时必接	是；完整	内置规则只将该事件计入同 IP 的尝试总数，无需 <code>attempted_account_hash</code> ；审计或宿主自定义规则需要时，可从认证结果补充不可逆哈希	成功事件不直接触发；仅后续登录失败命中 AUTH-02 时 <code>RATE_LIMIT</code>	成功事件作为聚合证据；控制执行仍保留审计记录
<code>auth:login-failure</code>	LOGIN_FAILURE	AUTH-01、AUTH-02、AUTH-03	认证能力存在时必接	是；需补充	<code>attempted_account_hash</code> 、 <code>account_status</code> ：参数或认证结果；缺少前者时仅按已认证主体或来源 IP 回退聚合	AUTH-01： <code>REQUIRE_CAPTCHA</code> 、 <code>RATE_LIMIT</code> ；AUTH-02： <code>RATE_LIMIT</code> ；AUTH-03： <code>DENY</code>	<code>account_status</code> 和认证结果来自服务端；自动动作保留审计证据
<code>auth:mfa-failure</code>	MFA_FAILURE	无默认规则	MFA 能力存在时必接	是；完整（无内置规则）	无内置规则必填事实；审计或宿主自定义规则可从参数或 MFA 结果补充 <code>attempted_account_hash</code> 与失败原因码	无默认控制；需宿主自定义规则请求控制	审核 MFA 失败次数、原因码与任何自定义控制
<code>session:concurrent</code>	SESSION_CONCURRENT	SESS-01	并发会话管理能力存在时必接	是；需补充	<code>dataCount</code> （并发会话数）、 <code>different_networks</code> ：会话评估后	<code>REQUIRE_MFA</code>	审核会话判定来源、网络差异和 MFA 执行结果
<code>access:denied</code>	ACCESS_DENIED 或 RESOURCE_SCOPE_DENIED	AUTHZ-01	资源域拒绝必接	是；完整	可信授权决定和服务端资源域拒绝结果；经 <code>ResourceAccessGuard</code> 记录	<code>RATE_LIMIT</code>	审核拒绝的授权证据；不得把参数映射作为授权结论
<code>resource:read</code>	ACCESS_ALLOWED 或 QUERY	AUTHZ-02	普通读取可选增强	是；需补充	<code>resourceId</code> 、 <code>sequential_access</code> ：参数路径、授权/查询结果或注册式草稿	<code>DENY</code> 、 <code>REVOKE_SESSION</code>	审核枚举判定、资源范围和会话撤销结果

建议 code		内置规则	宿主接入级别	自动注解支持	必需额外事实及采集点	自动控制	审核要求
data:query	QUERY	DATA-01、DATA-02	可选增强；批量查询能力存在时建议接入	是；需补充	resourceId: 参数路径；查询数量（dataCount）：仅执行后 Enricher 或注册式草稿；大量范围由 resourceId 集合聚合	RATE_LIMIT；DATA-02 可请求 DENY、REQUIRE_APPROVAL	审核查询范围、数据量与审批状态
data:sensitive-read	VIEW_SENSITIVE	DATA-02、DATA-03	敏感读取能力存在时必须接	是；需补充	resourceId: 参数路径；sensitive（静态属性）、work_hours、数据量：执行后或策略结果	DATA-02: DENY、REQUIRE_APPROVAL；DATA-03: REQUIRE_MFA	审核敏感分类、工作时段、审批及 MFA 结果
report:export	EXPORT	EXPT-01、EXPT-02	导出能力存在时必须接	是；需补充	resourceId: 参数路径；dataCount、baseline_ratio: 执行后；sensitivity: 静态属性或受控策略结果	EXPT-01: DENY、REQUIRE_APPROVAL；EXPT-02: DENY	审核导出范围、数量、基线和审批/拒绝结果
role:grant	ROLE_GRANT	PRIV-01、PRIV-02	权限授予能力存在时必须接	是；需补充	target_user_id、privilege_increase、high_privilege: 服务端授权结果或执行后	PRIV-01: DENY；PRIV-02: REQUIRE_MFA、REQUIRE_APPROVAL	审核操作者、目标用户、权限差异与审批
admin:create	ADMIN_CREATE	PRIV-02	管理员创建能力存在时必须接	是；需补充	目标管理员、high_privilege: 参数路径与创建后授权结果	REQUIRE_MFA、REQUIRE_APPROVAL	审核创建人、目标权限和审批
security:rule-change	RULE_CHANGE	SECU-01	规则管理能力存在时必须接	是；完整	可信变更主体、变更版本和服务端原因码；不要记录规则密钥或原始敏感配置	RECORD	审核变更单、审批人和版本差异
security:audit-config-change	AUDIT_CONFIG_CHANGE	SECU-01	审计配置能力存在时必须接	是；完整	可信变更主体、配置版本和服务端原因码	RECORD	审核变更单、审批人和生效范围
security:switch-change	SECURITY_SWITCH_CHANGE	SECU-01	安全开关能力存在时必须接	是；完整	可信变更主体、开关前后状态和服务端原因码	RECORD	审核变更单、审批人和回滚记录

控制策略独立于动作采集：**OBSERVE** 仍会记录事件、评估规则和创建告警，但不执行任何宿主控制。Starter 默认注册的 **DefaultControlActionTrigger** 是按 **ControlActionType** 逐项创建的安全回退：仅在 **ENFORCE** 命中对应规则动作且没有宿主实现时，**REQUIRE_CAPTCHA**、**RATE_LIMIT**、**LOCK**、**REVOKE_SESSION**、**REQUIRE_MFA**、**DENY** 和 **REQUIRE_APPROVAL** 才会产生 **SKIPPED**，原因码为 **DEFAULT_TRIGGER_REQUIRES_HOST_HANDLER:<ACTION>**。这让缺失接入可见且可审计，但绝不表示控制已经生效，也不能单独使 **ENFORCE** 启动。

触发分类	控制动作	Starter 默认实现	宿主必须接入	下的实际触发与审核边界
仅记录	RECORD	直接保留事件和告警证据，不注册控制触发器	否	自动记录，不改变业务请求；无需人工准许，但保留常规审计与告警处置。
自动宿主控制	REQUIRE_CAPTCHA、RATE_LIMIT、REVOKE_SESSION、REQUIRE_MFA、DENY	每个动作均有返回 SKIPPED 的默认回退	是，只要宿主允许对应内置规则产生实际控制，就必须实现同动作的幂等 ControlHandler 或 @ControlTrigger	只有宿主实现存在时才自动执行；不要求人工逐次准许，但每次执行和失败都必须审计，并按宿主运维流程复核。
人工审批	REQUIRE_APPROVAL	返回 SKIPPED 的默认回退	是，必须创建或更新待审批事项	可以自动创建待审批状态，不得自动放行；仅被授权人员人工批准后才能继续业务流程。
高风险锁定	LOCK	返回 SKIPPED 的默认回退	仅当宿主自定义规则需要锁定时必接	内置规则不产生锁定；自动锁定必须显式评审自定义规则、误报处置和解锁策略，默认按人工复核处理。

宿主实现同一动作即覆盖默认回退。Starter 将直接 **ControlHandler** 和 **@ControlTrigger** 适配器放在宿主优先层，再查默认层；若宿主要完全接管解析顺序，可自行提供一个 **ControlHandlerRegistry** Bean，Starter 会退让。由当前版本默认回退写入且带有框架回退标识的 **SKIPPED** 是临时接入状态：后续部署同动作宿主处理器后，下一次命中同一未关闭告警会执

行一次真实处理器并更新该控制记录；宿主自己返回的 `SKIPPED` 仍按普通幂等结果重放。早于该标识的历史 `SKIPPED` 记录也按普通幂等结果重放，宿主应按既有处置流程重新触发或关闭后重开告警，框架不会仅根据原因码猜测其来源。任意两个注解 Bean 重复绑定同一动作仍会在启动时失败，避免出现不确定的宿主处置。

3.2 MVC 注解采集

在控制器类型上标注 `@MonitorAction` 可设定默认动作；方法上的注解优先。全局拦截器只处理 Servlet MVC 的已标注 `HandlerMethod`，未标注路由不会产生动作事件。请求完成后的最终状态按以下优先级确定：控制器异常为 `FAILURE/HANDLER_EXCEPTION`；HTTP 401/403 为 `DENIED/HTTP_<状态码>`；其他 HTTP 4xx/5xx 为 `FAILURE/HTTP_<状态码>`；仅 HTTP 2xx/3xx 成功时才保留 `MonitorActionEnricher` 提供的业务 `result` 和 `reasonCode`，未提供 `result` 时默认 `SUCCESS`。适用的异常或 HTTP 终态优先于业务事实，且不保存异常消息。它不拦截 Service、消息消费者、定时任务、同类自调用或非 MVC 框架。

`@MonitorAction` 自身只定义启动时即可确定的动作元数据；`enrichers` 指定调用期动态事实的受控采集器。使用规则如下：

属性	是否必填	规则与示例
value	与 action 二选一	首选短写作动作编码，支持 <code>@MonitorAction("report:export")</code>
action	与 value 二选一	兼容既有调用；若两者都填写，内容必须完全一致
eventType	否	通用安全类别，默认 <code>QUERY</code> ；不要为每个业务动作另建枚举
resourceType	否	固定逻辑资源类别，例如 <code>report</code>
ruleTags	否	小写静态标签，例如 <code>sensitive-data</code> ；记录时映射为 <code>monitor.rule-tag.<tag>=true</code>
enrichers	否	<code>MonitorActionEnricher</code> 的类型数组；每个类型必须对应一个 Spring Bean

动作编码必须由服务端代码定义，建议采用 `域:动作`，例如 `report:export`。可重复的 `@MonitorActionAttribute` 让类型或方法声明稳定的非敏感属性，例如 `sensitivity=HIGH`；它和动作定义一样是静态事实。静态属性只从最终选中的动作源读取：方法自身带 `@MonitorAction` 时读取方法，否则读取类型；类型与方法的属性不会合并。

`@MonitorActionAttribute` 的参数默认值与声明位置相关，按下表使用：

参数	默认值	类型或方法声明（静态属性）	方法参数声明（动态事实）
target	ATTRIBUTE	必须保持 ATTRIBUTE	ATTRIBUTE 写入命名属性；需要写入资源或组织范围时显式指定 <code>RESOURCE_ID</code> 或 <code>ORG_SCOPE</code>
name	""	必须提供非空属性名	ATTRIBUTE 时必须提供非空属性名；其他目标不作为运行时值来源
value	""	必须提供非空静态值	必须保持为空，运行时值只从已绑定参数或 <code>path</code> 提取
path	""	必须保持为空	空值选择参数本身；非空值选择参数相对的受限 Java Bean 路径

写在方法参数上的同一注解只能提取运行时资源 ID、组织范围或已批准属性，不能写入事件类别、动作编码、IP、请求/追踪 ID、身份、角色、会话、服务端时间、数据量、最终结果或原因码。参数声明必须保持 `value` 为空；ATTRIBUTE 目标必须提供 `name`。`RESOURCE_ID` 和 `ORG_SCOPE` 的运行时值始终来自参数或其 `path`，不能把 `name` 或 `value` 当作运行时值。任何参数或 `Enricher` 提供的 `monitor.rule-tag.*` 属性都会被丢弃，不能新增或覆盖静态规则标签。

```
@RestController
@RequestMapping("/reports")
class ReportController {
    @MonitorAction(
        value = "report:export",
        eventType = SecurityEventType.EXPORT,
        resourceType = "report",
        ruleTags = {"sensitive-data", "bulk-operation"},
        enrichers = ReportExportFacts.class
    )
    @MonitorActionAttribute(name = "sensitivity", value = "HIGH")
    @PostMapping("/export")
    public ExportResult export(
        @MonitorActionAttribute(
            target = MonitorActionAttributeTarget.RESOURCE_ID,
            path = "report.id")
        @RequestBody ExportRequest request) {
        return reportService.export(request.getReport().getId());
    }
}

@Component
final class ReportExportFacts implements MonitorActionEnricher {
    @Override
    public MonitorActionFacts enrich(MonitorActionInvocation invocation) {
        if (invocation.getPhase() != MonitorActionInvocation.Phase.AFTER_RETURNING) {
            return MonitorActionFacts.empty();
        }
        ExportResult result = (ExportResult) invocation.getReturnValue();
        return MonitorActionFacts.builder()
            .dataCount(result.getRowCount())
            .reasonCode("EXPORT_COMPLETED")
            .build();
    }
}
```

控制器必须可被 Spring 代理，因此不能声明为 `final`。`ReportExportFacts` 必须是 Spring Bean；它可使用宿主服务，但只返回 `MonitorActionFacts`，不能直接改写草稿的可信字段。Enricher 会在 `BEFORE`、`AFTER_RETURNING` 与 `AFTER_THROWING` 三个阶段收到调用快照；采集器失败只放弃本次事实采集，不影响控制器响应。静态属性不可被参数映射或动态 Enricher 覆盖。参数路径只作为补充证据，不能替代身份、授权或业务结果。

参数 `path` 为空时直接选择已绑定的参数本身，适合标量路径变量；非空时是受限 Java Bean 路径，语法为 `identifier(.identifier|[nonnegative])*`，例如 `report.id`、`tenant.code`、`items[0].id`。它只能经过公开 Java Bean getter/字段，并且只允许数组或 `List` 的非负下标；禁止 `class`、方法调用、`Map` 任意键、静态成员、负下标、变量和任何表达式操作。每个属性路径段都按 `SecurityFieldSanitizer` 的同一凭据类关键字策略校验；包含该策略定义的凭据类关键字（例如 `password`、`token`、`cookie`、`secret`、`credential` 或 `authorization`）的段会在调用 `getter` 或读取字段前被拒绝。比较会忽略大小写和分隔符，因此 `privateKey`、`private_key`、`smsCode` 与 `sms_code` 等等价形式都会被拒绝。空路径以及非空路径的最终值只接受运行时精确类型为 `String`、`Boolean`、`Character`、`Byte`、`Short`、`Integer`、`Long`、`Float` 或 `Double` 的值；任意 `CharSequence/Number` 实现或子类、`Enum`、数组、`Map`、`Iterable`/集合和其他对象都会被跳过。值使用对应 JDK 类型的格式化方法生成，不会调用宿主对象的 `toString()` 或写入对象载荷。路径不存在、提取值为 `null`、下标越界或 `getter` 抛出异常时，只忽略该项事实采集，不

影响控制器业务或响应；空字符串仍是已提供的值，不会自动跳过。

事实的信息源必须按用途选择，不能以低可信度来源覆盖高可信度字段：

信息源	适用事实	使用方式
静态	固定 <code>sensitivity</code> 、 <code>high_privilege</code> 、资源类型、动作语义	类型/方法上的 <code>@MonitorActionAttribute</code> 与 <code>@MonitorAction</code>
参数路径	<code>resourceId</code> 、组织范围、批准的非敏感属性	参数上的 <code>@MonitorActionAttribute(path = "...")</code>
调用阶段	导出数量、时延、业务结果、稳定原因码，以及执行前后的已批准事实	<code>MonitorActionEnricher</code> 在 <code>BEFORE</code> 、 <code>AFTER_RETURNING</code> 、 <code>AFTER_THROWING</code> 返回 <code>MonitorActionFacts</code>
注册表/草稿	消息、任务、异步、流式或非 MVC 动作的完整动态事实	<code>MonitoringActionRegistry</code> 与 <code>ActionEventRecorder.draft(...)</code>

MVC 依赖是可选的：批处理、消息消费或非 `Servlet` 宿主不会加载 MVC 拦截器，也不需要引入 `spring-webmvc`。异步 `Callable`、`DeferredResult`、流式响应、返回签名含未解析 `TypeVariable`/通配符或未解析泛型 `HttpEntity/ResponseEntity` 的方法、`Service` 同类自调用和非 MVC 框架也不在自动采集范围内，即使运行时对象恰好同步也继续使用 `ActionEventRecorder.draft(...)`。`@ControlTrigger` 的控制绑定仍可被 Starter 发现。

`abnormal.access.monitor.instrumentation.enabled` 默认是 `true`，与 `frontend.enabled` 相互独立。即使关闭前端遥测，启用动作采集时仍会从可信代理、服务端身份提供器和请求元数据建立事件上下文。注解采集不会保存原始方法参数、请求体或响应体；它只提取已声明的受限事实，`Enricher` 也不得加入密码、令牌、`Cookie` 或原始载荷。

3.3 注册式方法调用埋点

为服务、消息和任务统一动作语义时，宿主应提供自己的 `MonitoringActionRegistry` Bean，以覆盖 Starter 的空注册表。`MonitorActionDefinition` 与 `@MonitorAction` 使用相同的 `action`、`eventType`、`resourceType`、规则标记和静态属性模型；注册表拒绝同一动作编码的冲突定义。

```
@Configuration
class MonitoringActionConfiguration {
    @Bean
    MonitoringActionRegistry monitoringActionRegistry() {
        return new MonitoringActionRegistry()
            .register(MonitorActionDefinition.builder("report:export")
                .eventType(SecurityEventType.EXPORT)
                .resourceType("report")
                .ruleTags("sensitive-data", "bulk-operation")
                .build());
    }
}
```

最简的服务记录可以直接调用 `record(action, request, identity, result, reasonCode)`。需要动态事实时先从记录器取得草稿，再补充字段并记录：

```
public ExportResult export(String reportId, MonitoringRequestContext request, IdentityContext identity) {
    long startedAt = System.currentTimeMillis();
    ExportResult result = exportService.run(reportId);

    recorder.record(recorder.draft("report:export", request, identity)
        .resourceId(reportId)
        .dataCount(result.getRowCount())
        .latencyMs(System.currentTimeMillis() - startedAt)
        .result(SecurityEventResult.SUCCESS)
        .reasonCode("EXPORT_COMPLETED")
        .attribute("sensitivity", "HIGH")
        .build());
    return result;
}
```

`draft(...)` 已预填充静态动作定义、可信 IP、请求 ID、追踪 ID、身份、角色、会话和服务端时间。调用方只应补充动态的、已经脱敏并获批准的业务事实，不应覆盖这些可信字段。动作未注册时 `record(String, ...)` 与 `draft(String, ...)` 会失败，这是为了防止多个调用点对同一动作赋予不同类别。

消息消费或任务可显式构造 `MonitoringRequestContext`，其中 `method`、`path`、`sourceIp` 和 `requestId` 为必填；例如使用 `method("MESSAGE")`、主题路径、可信 broker 标识和消息 ID。将上游链路标识放入 `traceId(...)`，以便事件与日志查询关联。

3.4 原始草稿与 `EventEnricher`

只有非标准适配器确实无法使用 `ActionEventRecorder` 时，才直接构造 `SecurityEventDraft`。该模型至少要求 `eventType`、`action`、`result`、`sourceIp`、`requestId` 和 `occurredAt`，旧示例中只填写动作和结果是不完整的：

```
SecurityEventDraft draft = SecurityEventDraft.builder()
    .eventType(SecurityEventType.EXPORT)
    .action("report:export")
    .result(SecurityEventResult.SUCCESS)
    .sourceIp(trustedRequest.getSourceIp())
    .requestId(trustedRequest.getRequestId())
    .traceId(trustedRequest.getTraceId())
    .occurredAt(Instant.now())
    .resourceType("report")
    .resourceId(reportId)
    .dataCount(exportedRows)
    .build();
monitor.record(draft);
```

`EventEnricher` 是原始草稿的显式扩展点，不是 MVC 自动回调点；不要把它与 `@MonitorAction(enrichers = ...)` 使用的 `MonitorActionEnricher` 混淆。若宿主在安全事件监听器、消息适配器或审计桥接器中实现前者，应在已完成安全审查后显式调用，并记录返回值：

```
SecurityEventDraft enriched = eventEnricher.enrich(draft, trustedRequest, identity);
recorder.record(enriched);
```

监听补充只能增加经审批的业务摘要，不能改写适配器已经确定的可信主字段。建议的可补充字段如下：

字段	可补充内容与可信来源
resourceType、resourceId	已解析的业务资源类别和稳定资源标识；来自服务端路由、授权结果或执行结果，而不是浏览器声明。
orgScope	租户、组织或数据域边界；来自服务端身份、授权或资源查询结果。
dataCount、latencyMs	已执行操作的影响数量和服务端实测耗时；用于导出、批量查询、会话并发等规则。
reasonCode	稳定、非敏感的业务原因码，例如 EXPORT_COMPLETED、APPROVAL_PENDING；不可写异常原文。
attributes	动作相关的受控摘要，例如 attempted_account_hash、account_status、different_networks、sequential_access、baseline_ratio、sensitivity、work_hours、target_user_id、privilege_increase、high_privilege、approval_state 和版本标识。账号类标识应使用不可逆散列，属性必须来自服务端结果或受控策略。

eventType、action、result、sourceIp、requestId、traceId、身份/角色、会话或设备散列以及 occurredAt 均由适配器确定，Enricher 必须原样保留。属性键会清洗并限制为 128 个字符，值限制为 512 个字符；含 password、token、cookie、secret、authorization 等凭据含义的键会被拒绝。任何密码、令牌、Cookie、原始请求/响应体、原始敏感数据或未经批准的业务字段都不得加入。

若控制必须发生在业务副作用之前，先执行宿主授权/控制或在该位置显式调用监测接口；完成后的 MVC 注解事件只用于解耦审计与规则分析，不能充当前置拦截器。

3.5 规则控制方法绑定

无需实现 `ControlHandler` 也可在任意 Spring Bean 的公开方法上声明控制绑定。Starter 会发现该方法，并用框架无关的 `AnnotatedControlHandler` 适配到核心控制服务：

```
@Component
final class HostControlActions {
    @ControlTrigger (ControlActionType.RATE_LIMIT)
    public ControlExecution rateLimit (ControlCommand command) {
        hostRateLimiter.apply (command.getSubject(), command.getExpiresAt());
        return ControlExecution.succeeded (command.getIdempotencyKey());
    }
}
```

绑定方法必须只接收一个 `ControlCommand`，返回 `void` 或 `ControlExecution`。任意注解 Bean 中同一动作的重复绑定、非公开方法或不匹配签名都会在绑定时失败。处理方法必须用 `idempotencyKey` 去重，且只能标注真正可执行的非 `RECORD` 动作；空绑定不能使 `ENFORCE` 启动。已有 `ControlHandler` 实现仍可与注解绑定并存：宿主实现优先于 `DefaultControlActionTrigger`，而直接注册的 `ControlHandler` 会先于注解适配器解析。若 `ENFORCE` 命中规则却没有宿主实现，默认回退会保存 `SKIPPED`，提示补齐该动作，而不会假装执行控制；后来补齐同动作处理器后，下次命中同一未关闭告警会重试由当前版本默认回退生成的临时记录。

4. 内部不可变规则与持久化动态规则

规则有两个独立通道，管理界面和运行时不得混用：

维度	内部规则	持久化动态规则版本
来源	DefaultRuleCatalog、宿主 DetectionRule Bean、InternalRuleContributor	security_rule 表
注册时机	应用启动、创建 SecurityMonitor 前	管理端插入版本或切换启停
运行时可变性	InternalRuleRegistry.freeze() 后不可变	数据库记录可新增版本、查询和调整 enabled
管理查询	InternalRuleRegistry.entries(), 固定 source=INTERNAL、mutable=false	MonitoringAdministrationMapper.findRuleVersions(), 来源为 PERSISTED
是否自动参与当前运行规则	是，冻结快照传给监测器	否；当前组件不自动把表定义编译或装载为 DetectionRule

使用 `InternalRuleContributor` 贡献随代码发布的合规基线或特定动作规则。`Starter` 会先注册默认规则 and 所有 `DetectionRule` Bean，再执行贡献者，随后冻结注册表；重复规则 ID 或冻结后的注册都会失败。

```
@Bean
InternalRuleContributor reportInternalRules() {
    return registrar -> registrar.register(new ReportExportRule());
}

// ReportExportRule implements DetectionRule, 并提供稳定且唯一的 ruleId。
```

管理端应分别读取 `internalRuleRegistry.entries()` 与已打开 `SqlSession` 中的 `MonitoringAdministrationMapper.findRuleVersions()`，再按 `source/mutable` 展示。
`setRuleEnabled(ruleId, version, enabled)` 只更新该持久化版本的管理标记，不能修改内部规则，也不会立即改变当前 `DefaultSecurityMonitor` 的规则快照。动态规则若要生效，宿主必须单独实现经过审批、语法校验、版本选择和灰度发布的加载器。

5. MyBatis 迁移与告警闭环

生产环境必须提供数据源和 `SqlSessionFactory`，并把 JAR 中的 `db/monitoring-schema.sql` 纳入 Flyway、Liquibase 或既有迁移流程。`Starter` 检测到 `SqlSessionFactory` 后注册 `Mapper` 与 `InstantTypeHandler`，使用 `MyBatisMonitoringRepository`；否则使用内存仓储，仅适合本地开发和测试。组件不会自动建表。

迁移包含 `security_event`、事件角色/属性表、`security_rule`、`security_alert`、`alert_event_link`、`control_action`、`alert_disposition` 和 `security_whitelist`。迁移账户需要建表建索引权限；运行账户只保留新增、查询和必要更新权限。

通过 `AlertLifecycleService` 进行 `acknowledge`、`startInvestigation`、`close` 或 `falsePositive`。每次状态转换都会追加 `alert_disposition`；原始安全事件不应被覆盖或删除。关闭告警需要证据摘要。

6. 前端补充信息统一接入

前端补充信息必须先到达宿主系统的后端 HTTP API，再由宿主适配后调用组件。组件统一接收、校验、规范化与记录功能，以及 `FrontendSignal` v1 的语义；组件不提供通用 `Controller`、不规定固定 URL、不实现认证，也不定义响应 JSON。宿主可推荐使用 `POST` 与 `application/json; /api/monitoring/frontend-signals` 仅是非规范性示例，实际路由、认证、响应模型和框架实现均由宿主负责。

组件的统一入口为 `FrontendSignalRecorder.record(signal, serverContext)`。浏览器提交的值仅是非权威的补充证据，不能替代服务端认证、授权或业务判定。

宿主可以继续使用自己的原生请求 DTO，但必须做显式、逐字段映射：`pageCode` -> `route`、`operationCode` -> `action`、单向哈希后的订单 ID -> `resource_id_hash`。未映射的业务字段不得放入 `attributes` 或任意扩展 `payload`。要发布新字段时，必须同时更新 `Schema`、`FrontendSignal` 白名单、测试和本文档。

6.1 标准请求契约与字段映射

标准 Schema 位于 `web-contract/src/main/resources/frontend-signal.schema.json`，当前版本为 1.0。以下字段均来自浏览器，除明确由服务器验证或派生的关联信息外，均只能作为非权威补充证据。

字段	必填	映射规则	信任级别
<code>contract_version</code>	否，存在时必须为 1.0	原样映射；未提供时 v1 默认按 1.0 处理	非权威补充证据
<code>client_event_id</code>	是	浏览器为一次前端事件生成的 UUID 或等价唯一标识	非权威补充证据
<code>occurred_at</code>	是	浏览器事件时间，解析为 UTC 时间戳	非权威补充证据，必须做时钟偏差校验
<code>request_id</code>	是	关联宿主服务器已签发或可验证的请求标识	非权威关联证据；不建立任何身份
<code>trace_id</code>	否	映射宿主允许前端回传的链路关联标识	非权威补充证据
<code>route</code>	是	原生 DTO 的 <code>pageCode</code> 显式映射为稳定路由语义	非权威补充证据
<code>action</code>	是	原生 DTO 的 <code>operationCode</code> 显式映射为稳定操作语义	非权威补充证据
<code>device_id_hash</code>	否	仅映射经单向哈希的设备标识	非权威补充证据
<code>resource_type</code>	否	映射受控的资源类别，例如 <code>order</code>	非权威补充证据
<code>resource_id_hash</code>	否	单向哈希后的订单 ID 等业务资源标识显式映射至此	非权威补充证据
<code>attributes</code>	否	仅映射已发布的低风险交互属性	非权威补充证据

`device_id_hash` 与 `resource_id_hash` 出现时都必须以 `sha256:` 开头。`attributes` 仅允许 `feature`、`interaction`、`page_type`、`network_type`、`ui_version`、`resource_class`、`outcome_hint`；对象及其嵌套 `attributes` 均由 `additionalProperties=false` 拒绝未知字段。`request_id` 必须关联到服务器已签发或可验证的请求，但它不建立用户、会话或任何身份。

```
{
  "contract_version": "1.0",
  "client_event_id": "fd584fda-3d98-4e73-857f-a19e3be1e456",
  "occurred_at": "2026-07-23T09:30:00Z",
  "request_id": "req-5af1f1f0",
  "trace_id": "trace-41c92bd9",
  "route": "/orders/detail",
  "action": "order:view",
  "resource_type": "order",
  "resource_id_hash": "sha256:6a440d8653c80e2176c0f59ec1d4c572e0ebd4b4667c491505c9067cfde2a8c4",
  "attributes": {
    "feature": "order-center",
    "page_type": "detail",
    "ui_version": "2026.07"
  }
}
```

浏览器不得提交或断言用户 ID、角色、账户类型、会话 ID、来源 IP、风险等级、告警 ID、控制动作、授权决定或最终业务结果。这些信息只能来自宿主已认证上下文、可信代理配置或服务器侧业务处理。

6.2 宿主接口适配流程

下面是伪代码。`HostFrontendSignalRequest`、`HostAuthenticationContext`、`Schema` 校验器、请求 ID 验证器、接入限流器、映射函数和响应工厂均为宿主自有类型；示例路径也不构成组件的固定端点。

```
// 伪代码：路径、DTO、认证类型、校验器、请求 ID 验证器、限流器、映射函数和响应工厂都由宿主系统定义。
@PostMapping(
    path = "/api/monitoring/frontend-signals",
    consumes = MediaType.APPLICATION_JSON_VALUE)
public Object receiveFrontendSignal(
    @RequestBody HostFrontendSignalRequest request,
    HostAuthenticationContext authenticationContext,
    HttpServletRequest httpRequest) {
    // 控制器前的安全链已完成认证和端点授权；Cookie/会话认证必须先完成 CSRF 防护。
    // 若支持跨源浏览器访问，宿主另行配置明确的 CORS 来源白名单和凭据策略；CORS 绝不替代 CSRF。

    String trustedSourceIp = trustedProxyResolver.resolveClientIp(
        httpRequest.getRemoteAddr(), httpRequest.getHeader("X-Forwarded-For"));
    // 可信来源 IP 仅由 TrustedProxyResolver 解析，阻止浏览器伪造转发头绕过基于 IP 的防护。
    HostIngressRateLimitResult rateLimit = hostIngressRateLimiter.check(
        authenticationContext.getTrustedUserOrSessionKey(), trustedSourceIp);
    // 宿主限流器同时以已认证用户或会话与可信 IP 为键，先限制恶意流量再消耗 Schema 或映射资源。
    if (!rateLimit.isAllowed()) {
        // 拒绝必须立即停止处理，并由宿主自己的响应工厂返回 HTTP 429。
        return hostTooManyRequestsResponse();
    }

    hostFrontendSignalSchemaValidator.validate(request);
    // Schema 拒绝未知字段和不支持版本；原始请求体、Cookie、令牌和业务实体绝不写入监控事件。

    HostRequestIdVerification requestIdVerification = hostRequestIdVerifier.verify(
        request.getRequestId(), authenticationContext, httpRequest);
    // 宿主验证浏览器提交的关联 ID 是否匹配服务器已签发或当前请求可验证的值；不匹配即拒绝并返回宿主 HTTP 400。
    if (!requestIdVerification.isValid()) {
        return hostBadRequestResponse();
    }
    String verifiedRequestId = requestIdVerification.getVerifiedRequestId();

    FrontendSignal signal = FrontendSignal.builder()
        .contractVersion(request.getContractVersion())
        .clientEventId(request.getClientEventId())
        .occurredAt(request.getOccurredAt())
        .requestId(verifiedRequestId)
        .traceId(request.getTraceId())
        .route(mapPageCodeToRoute(request.getPageCode()))
        .action(mapOperationCodeToAction(request.getOperationCode()))
        .deviceIdHash(request.getDeviceIdHash())
        .resourceType(mapResourceType(request.getResourceType()))
        .resourceIdHash(hashOrderIdOneWay(request.getOrderId()))
        .attribute("feature", request.getFeature())
        .attribute("page_type", request.getPageType())
        .attribute("ui_version", request.getUiVersion())
        .build();
    // 原生 DTO 只通过显式映射进入 FrontendSignal，由其强制长度、哈希前缀和 attributes 白名单。
    Instant receivedAt = clock.instant();
```

```
frontendSignalValidator.validate(signal, receivedAt);
// clock.instant() 是权威接收时间; FrontendSignalValidator 的时钟偏差失败应由宿主转换为 HTTP 400。

FrontendServerContext serverContext = FrontendServerContext.builder()
    .sourceIp(trustedSourceIp)
    .userId(authenticationContext.getAuthenticatedUserId())
    .sessionIdHash(authenticationContext.getTrustedSessionIdHash())
    .accountType(authenticationContext.getAccountType())
    .roleIds(authenticationContext.getRoleIds())
    .build();
// 用户、角色和会话哈希只从已认证上下文派生, IP 复用上方 TrustedProxyResolver 得到的可信值。

frontendSignalRecorder.record(signal, serverContext);
// 记录器将前端补充证据与可信服务器上下文合并, 供组件后续规则处理。
return hostSuccessResponse();
// 成功响应保持为宿主自己的 API 模型, 组件不提供任何响应 JSON。
}
```

处理顺序必须严格为：认证和端点授权 -> Cookie/会话认证时的 CSRF 防护 -> 若支持跨源浏览器访问，单独执行明确的 CORS 来源白名单和凭据策略（CORS 绝不替代 CSRF） -> JSON Content-Type -> 请求体大小限制 -> 对可信用户和可信 IP 限流 -> JSON Schema -> 宿主 request_id 验证 -> FrontendSignal 构造校验 -> 时钟偏差 -> 可信服务器上下文 -> FrontendSignalRecorder.record。只有直接来源已被信任时，且仅可由 TrustedProxyResolver 解释转发请求头。

6.3 响应语义、重试与验收

组件不提供响应 JSON、错误或异常包装器。宿主保留自己的 API 响应体；HTTP 语义建议如下：无效 JSON、未知字段、不支持的契约版本、对象校验失败或时钟偏差为 400；未认证为 401；无权访问为 403；请求体过大为 413；触发限流为 429；不可恢复的内部错误为 5xx。错误响应不得回显完整事件、Cookie、会话值、规则、风险分数、告警详情或控制信息。

宿主可选用短期去重键：可信用户或会话范围 + client_event_id。该组件不提供端到端去重；重试、幂等窗口和最终响应语义仍由宿主 API 决定。

验收场景	期望结果
标准提交	通过 Schema、构造校验和时钟偏差校验后，以可信服务器上下文调用 FrontendSignalRecorder.record。
原生 DTO 映射	pageCode、operationCode 和单向哈希订单 ID 分别明确映射到 route、action、resource_id_hash，未映射业务字段不被记录。
未知字段	顶层或 attributes 中存在未发布字段时，由 additionalProperties=false 拒绝并返回宿主 400。
时钟偏差	occurred_at 超过允许偏差时，不调用记录器，宿主返回 400。
身份/IP/角色伪造	浏览器声明被忽略或拒绝；身份、角色和 IP 仅从认证上下文及可信代理配置派生。
重复 client_event_id	在宿主配置的短期去重窗口内，按可信用户或会话范围处理；组件不承诺端到端去重。
限流	基于可信用户与可信 IP 执行限流，超过阈值时宿主返回 429，不进入记录器。

7. 初始化插件


```
mvn io.github.jasperbigsum:commits:sys-abnormal-access-monitoring-maven-plugin:${abnormal-access-monitoring.version}:initialize `
'-Dabnormal.access.monitor.outputDirectory=src/main/resources/abnormal-access-monitoring' `
'-Dabnormal.access.monitor.packageName=com.example.orders.monitoring' `
'-Dabnormal.access.monitor.systemId=order-service'
```

参数	默认值	说明
abnormal.access.monitor.outputDirectory	\${project.basedir}/src/main/resources/abnormal-access-monitoring	模板输出目录
abnormal.access.monitor.packageName	com.example.monitoring	生成 Java 模板的包名
abnormal.access.monitor.systemId	change-me	生成 YAML 中的系统标识

目标创建以下 5 个文件：

文件	用途
application-abnormal-access-monitoring.yml	含 OBSERVE、instrumentation 和 mdc 安全默认值的组件配置
host-spi/HostMonitoringSpi.java	IdentityContextProvider、ResourceScopeAuthorizer、TrustedProxyResolver 的安全占位实现
host-spi/HostMonitoringActions.java	MonitoringActionRegistry Bean 与 report:export 注册式动作模板
host-spi/HostControlHandler.java	可执行控制动作的保守占位实现
frontend-signal-v1.example.json	前端信号契约示例

Java 模板需要移入宿主 `src/main/java`，并用真实认证、授权、代理信任、动作注册与幂等控制实现替换占位逻辑。所有文件以 `CREATE_NEW` 写入，已有文件只会保留并在 `Maven` 日志中提示。

8. 最小验收、升级与发布

在切换为 `ENFORCE` 前，至少完成下列最小验收：

场景	操作	预期结果
注解短写	调用 <code>@MonitorAction("report:read")</code> 的 MVC 方法	请求完成后产生 <code>QUERY</code> 事件，动作编码为 <code>report:read</code>
注解一致性	同时配置 <code>value</code> 和 <code>action</code>	两者不一致时应用初始化/首次解析失败；一致时只产生一个统一动作
动态动作事实	用 <code>recorder.draft("report:export", ...)</code> 补充资源 ID、数据量和时延	事件保留注册的静态定义与可信上下文，并保存补充的非敏感事实
非 MVC 入口	消息或任务传入自己的 <code>MonitoringRequestContext</code> 与 <code>IdentityContext</code>	不依赖 MVC 拦截器，也能用已注册动作产生标准事件
内部规则	注册 <code>InternalRuleContributor</code> 或 <code>DetectionRule</code> Bean	规则出现在 <code>entries()</code> 中且启动后不能修改
持久化版本	查询/切换 <code>security_rule</code> 的 <code>enabled</code>	管理视图更新；运行规则不会在没有宿主动态加载器时自动变化
MDC	开启 <code>mdc.enabled</code> ，发起带追踪头的请求	事件 <code>traceId</code> 与配置键对应的日志 <code>MDC</code> 值一致，完成请求后恢复原 <code>MDC</code>

场景	操作	预期结果
控制绑定	触发已绑定的 <code>@ControlTrigger</code> 动作	宿主方法收到唯一 <code>ControlCommand</code> ，按 <code>idempotencyKey</code> 去重
默认触发器替换	同时存在同动作的宿主 <code>ControlHandler</code> 或 <code>@ControlTrigger</code>	宿主实现优先，未接入的动作仅留下 <code>SKIPPED</code> 审计记录；默认回退不能使 <code>ENFORCE</code> 启动

升级组件前，先在预发布环境执行新迁移并用 `OBSERVE` 对照现有审计结果。由 Boot 2 升级至 Boot 3 时，替换 Starter、将宿主 MVC/Servlet 集成从 `javax.*` 迁移到 `jakarta.*`，并以 JDK 17 编译。变更规则阈值、控制 TTL、Schema 或 SPI 时，补充宿主验收用例并执行：

```
mvn clean verify -DskipTests=false
```